Pthreads Implementation

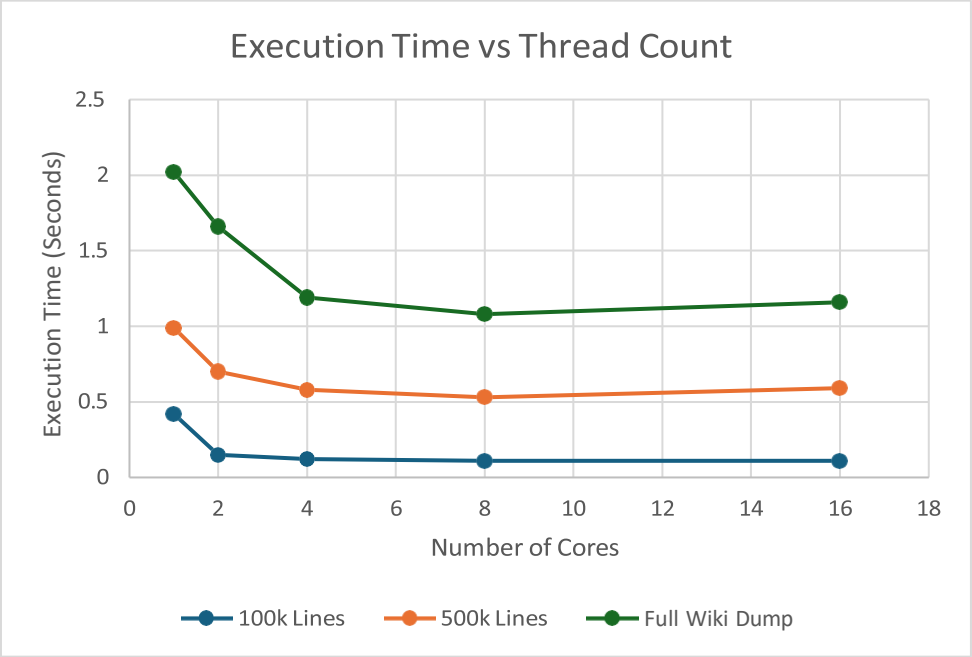
Software Architecture

The pthreads version of this application loads wiki dump and processes it in 5000 line chunks. The workload of the batch splits the load among active threads. Each thread calculates its chunk size and computes its specific start and end loops based on its thread id. The main thread uses pthread_join to wait for all threads to complete their chunks before printing the formatted output and goes to the next 5000 lines.

Performance Analysis

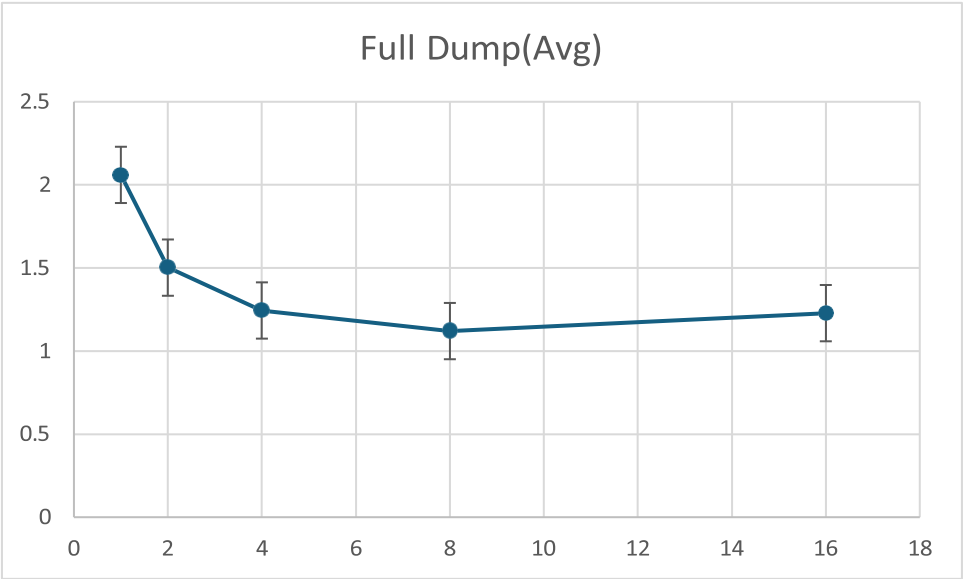
The initial performance analysis was gathered using the wiki_dump across 1, 2, 4, 8, and 16 threads.

Cores	100k Lines	500k Lines	Full Wiki Dump
1	0.42	0.99	2.02
2	0.15	0.7	1.66
4	0.12	0.58	1.19
8	0.11	0.53	1.08
16	0.11	0.59	1.16



Standard Deviation on Full Wiki Dump

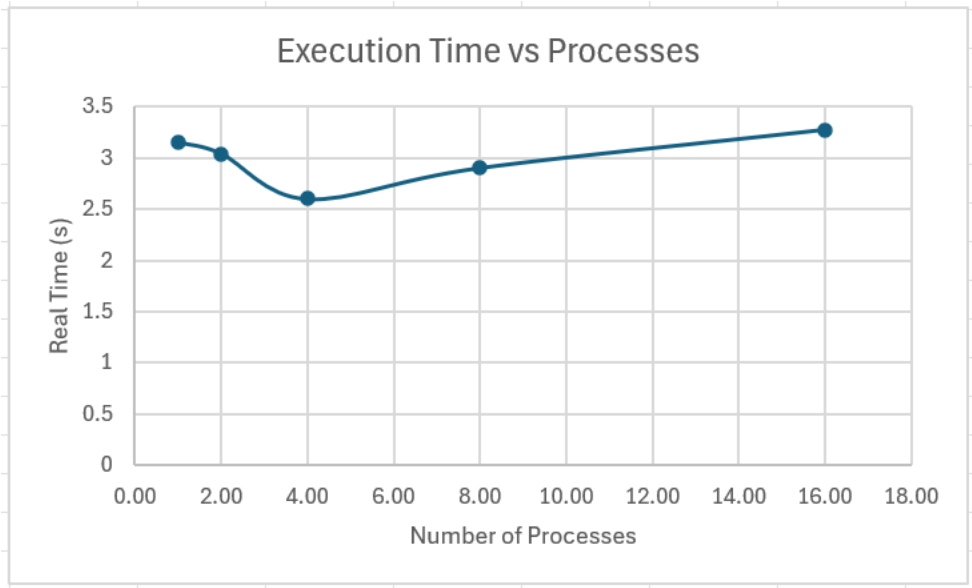
Cores	Average	Standard Deviation
1	2.06	0.079056942
2	1.502	0.004472136
4	1.244	0.005477226
8	1.12	0
16	1.228	0.016431677



Execution times were recorded across 5 runs. Standard deviation across all thread configurations remained below 0.08

MPI Implementation

The MPI implementation is very similar to the pthreads implementation in that it loads chunks and send it off to worker processes. However, instead of loading it into different threads, the program does it via processes. The processes then send data back to the master process which then aggregates data nicely. The pthreads utilizes global variables, while the MPI implementation uses local variables and sends and receives them between each other to corresponding local variables of the same type.



This graph shows the effect of the number of processes and the amount of time it takes to execute. The MPI implementation was compiled with 4 processes in mind, which is why the execution time increases with process count > 4.

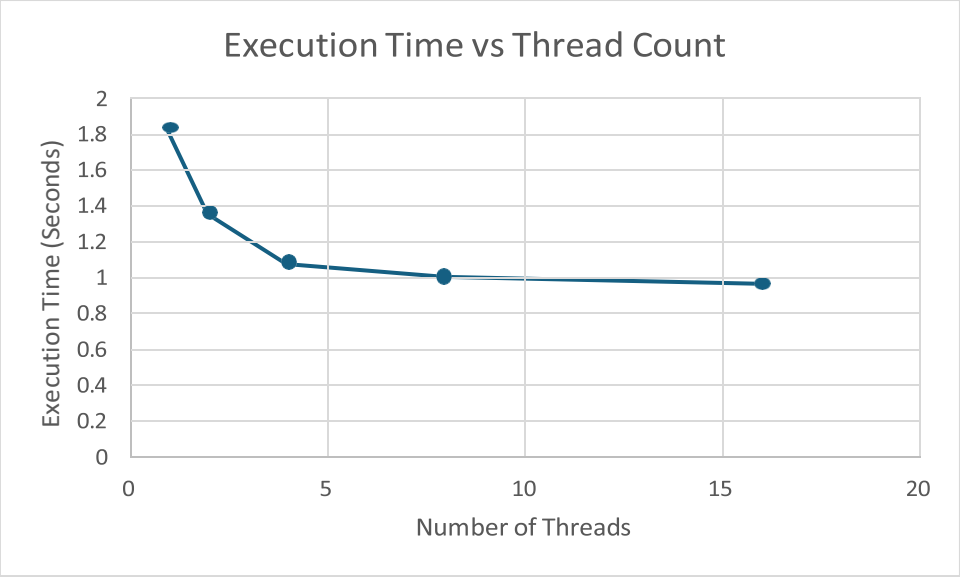
OpenMP Implementation

The OpenMP version of this application loads the wiki dump and processes it in chunks, like the other implementations. Instead of manually managing threads and workload, OpenMP handles thread creation and scheduling. Each batch is processed independently. The main thread waits implicitly at the end of the parallel region before printing results.

Performance:

The initial performance analysis was gathered using the wiki_dump across 1, 2, 4, 8, and 16 threads.

Threads	Execution Time
1	1.82
2	1.36
4	1.08
8	1.01
16	0.97



These results show improvements when increasing thread count until after 8 threads, when gains diminished. OpenMP does not scale across multiple nodes.

Threads	Nodes	Execution Time
8	1	1.01
8	2	1.01

Memory allocation also did not make a significant difference in runtime.